Task 8: Configure Azure Application Gateway for Two Linux Virtual Machines

Objective

To deploy web applications on two Ubuntu Linux Virtual Machines using Nginx and configure an Azure Application Gateway to route and manage incoming web traffic to backend virtual machines.

Environment Details

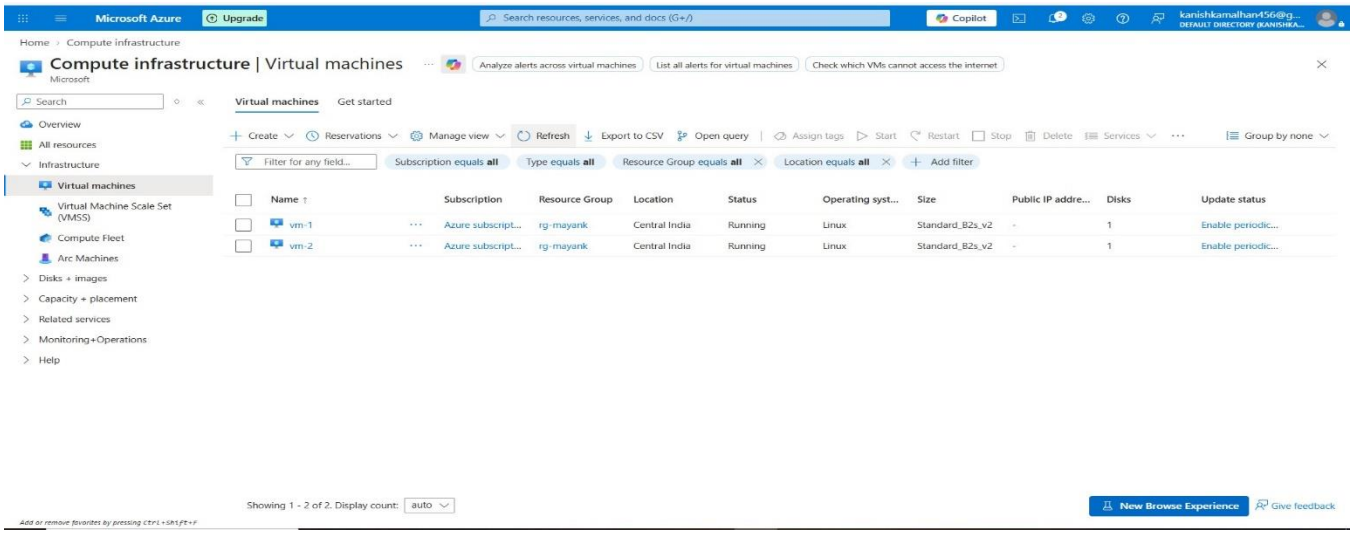
- Azure Subscription
- Resource Group: **rg-mayank**
- Ubuntu Linux VM-1
- Ubuntu Linux VM-2
- Virtual Network (VNet)
- Dedicated Application Gateway Subnet
- Nginx Web Server
- GitHub Web Applications
- Azure Application Gateway Standard V2

Step 1: Verify Virtual Machines

Verified that both Ubuntu Linux virtual machines were successfully deployed and running.

Observation

- VM-1 and VM-2 were in running state.
- Both virtual machines were ready for application deployment.
- Both servers would be used as backend targets for Application Gateway.



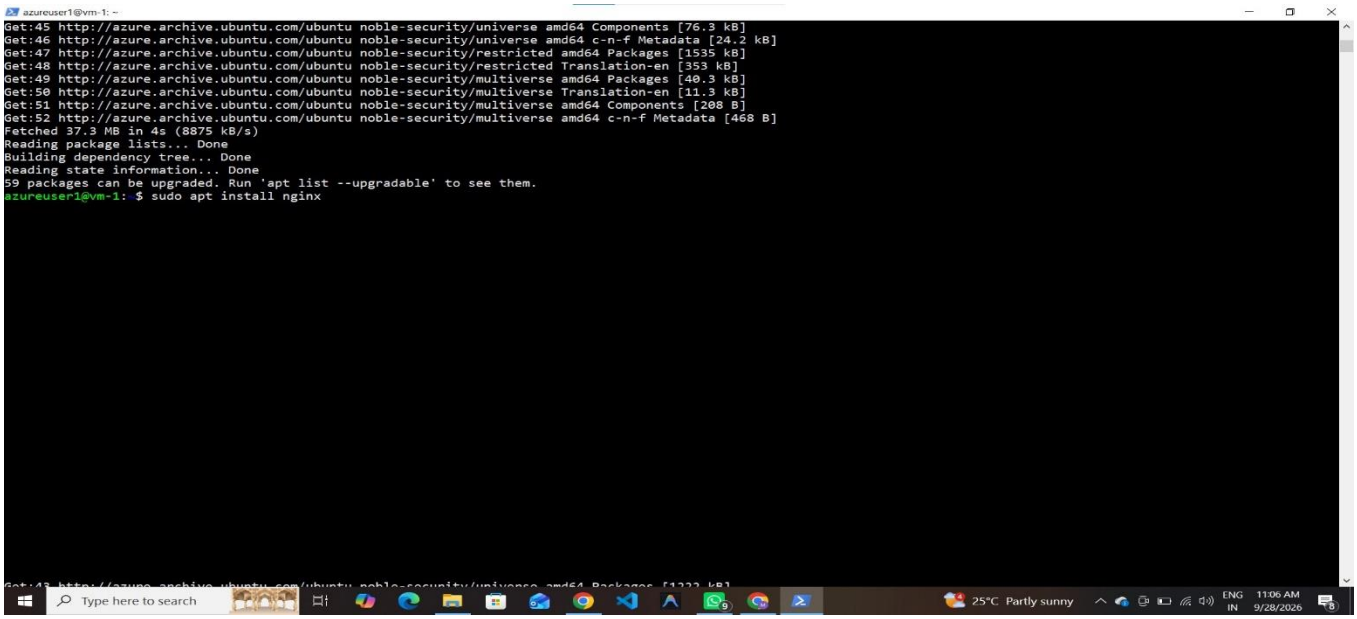
Step 2: Install Nginx on VM-1

Connected to VM-1 using SSH and installed the Nginx web server.

sudo apt update
sudo apt install nginx –y

Observation

- Package repository updated successfully.
- Nginx web server installed successfully.
- VM-1 was prepared for hosting web applications.



Step 3: Verify Nginx Web Root Directory

Navigated to the default Nginx document root directory.

cd /var/www/html
ls

Observation

- Default Nginx web page was available.
- The Nginx document root directory was verified successfully.
- The directory was ready for web application deployment.



Screenshot 3: Nginx Web Root Directory Verification

Step 4: Verify Application Deployment on VM-1

Verified the application files deployed inside the Nginx web root directory.

cd /var/www/html
ls

Observation

- Application files were successfully deployed on VM-1.
- Website resources including HTML, images, and assets were available.
- VM-1 was ready to serve application traffic.



Step 5: Verify Application Deployment on VM-2

Verified the web application files deployed on the second backend server.

cd /var/www/html
ls

Observation

- Application files were successfully deployed on VM-2.
- Required website resources were available.
- VM-2 was successfully configured as a backend web server.



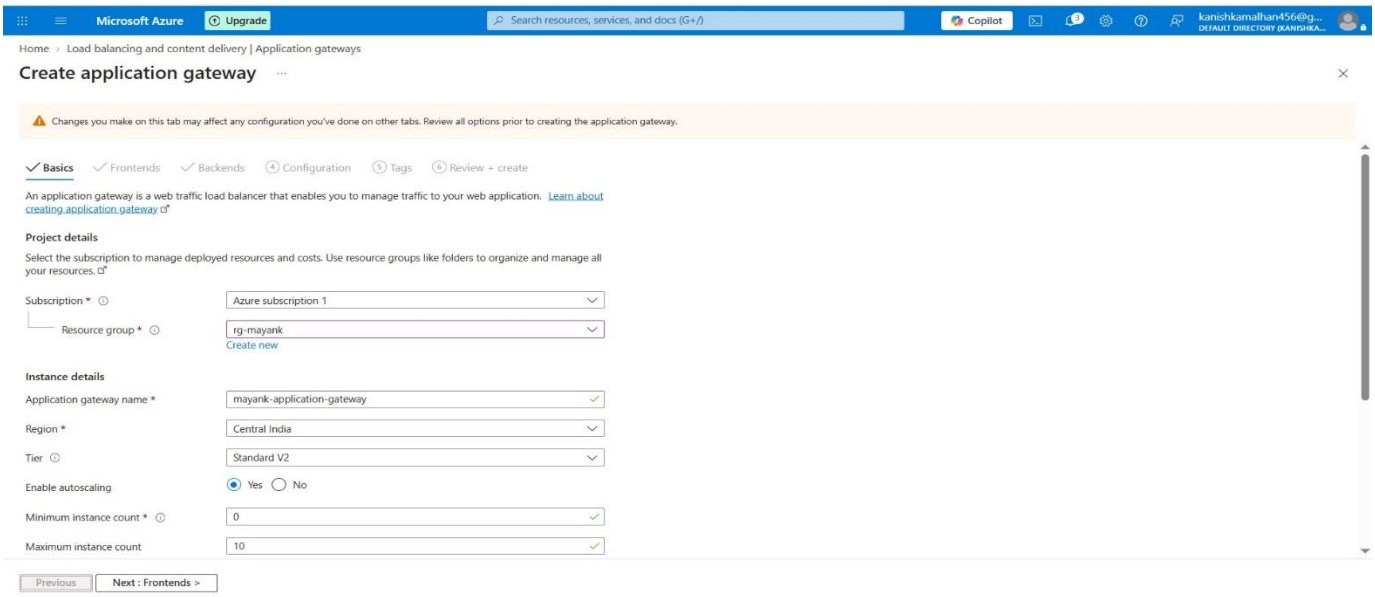
Step 6: Create Azure Application Gateway

Navigate to:

Azure Portal
→ Create Resource
→ Application Gateway

Observation

- Application Gateway deployment wizard started successfully.
- Standard V2 tier selected for improved scalability and performance.
- Autoscaling enabled to automatically handle incoming traffic.

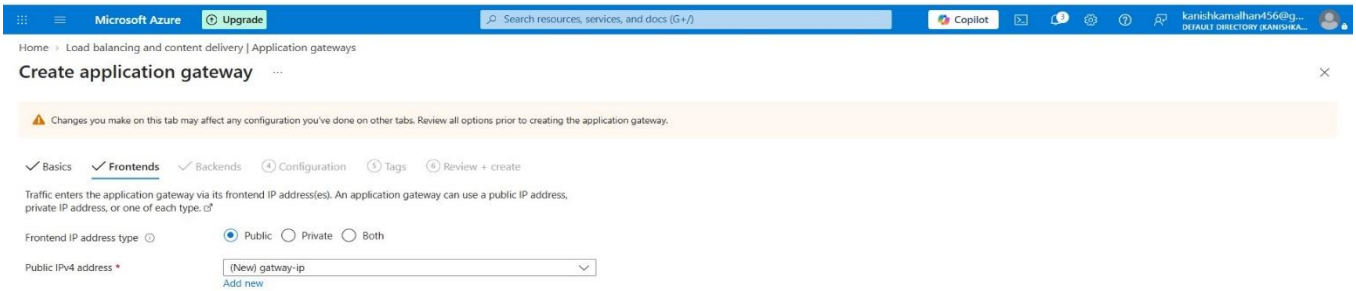


Step 7: Configure Frontend IP Address

Configured a public frontend endpoint for receiving external web traffic.

Observation

- Public frontend IP address configured successfully.
- Internet traffic can reach the Application Gateway through the assigned public endpoint.
- Frontend configuration completed successfully.



Step 8: Configure Backend Pool

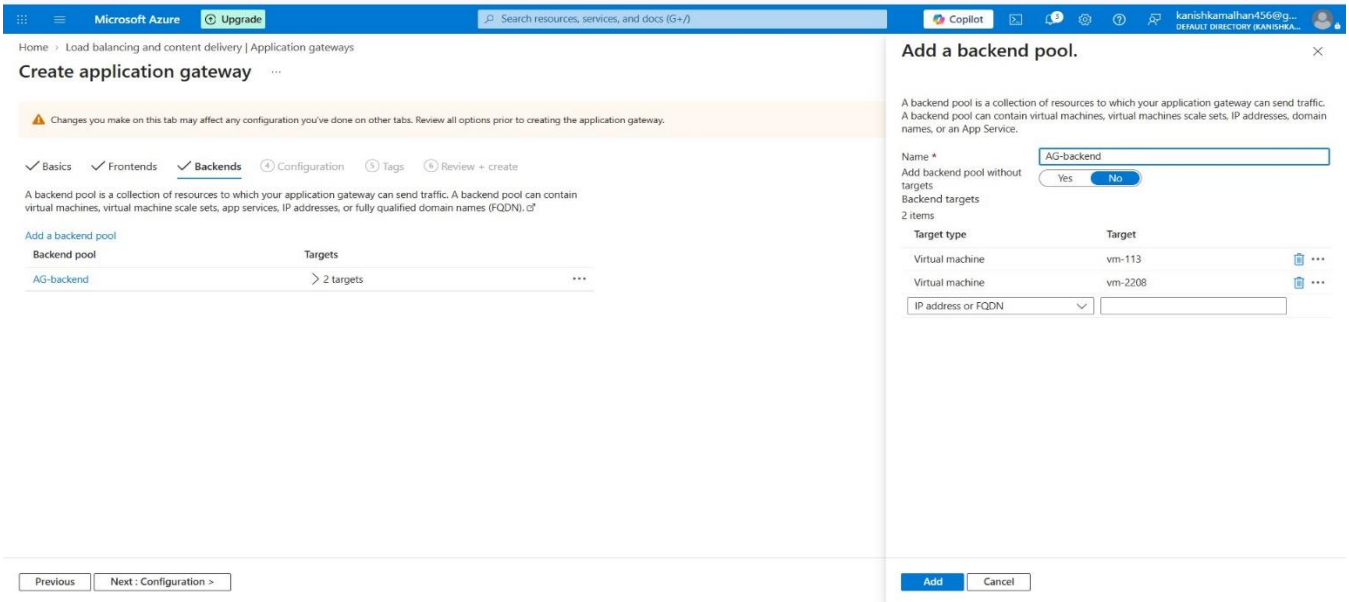
Created a backend pool and added both virtual machines as backend targets.

Backend Resources

VM-1
VM-2

Observation

- Backend Pool created successfully.
- Both backend virtual machines added successfully.
- Application Gateway can now route traffic to both backend servers.

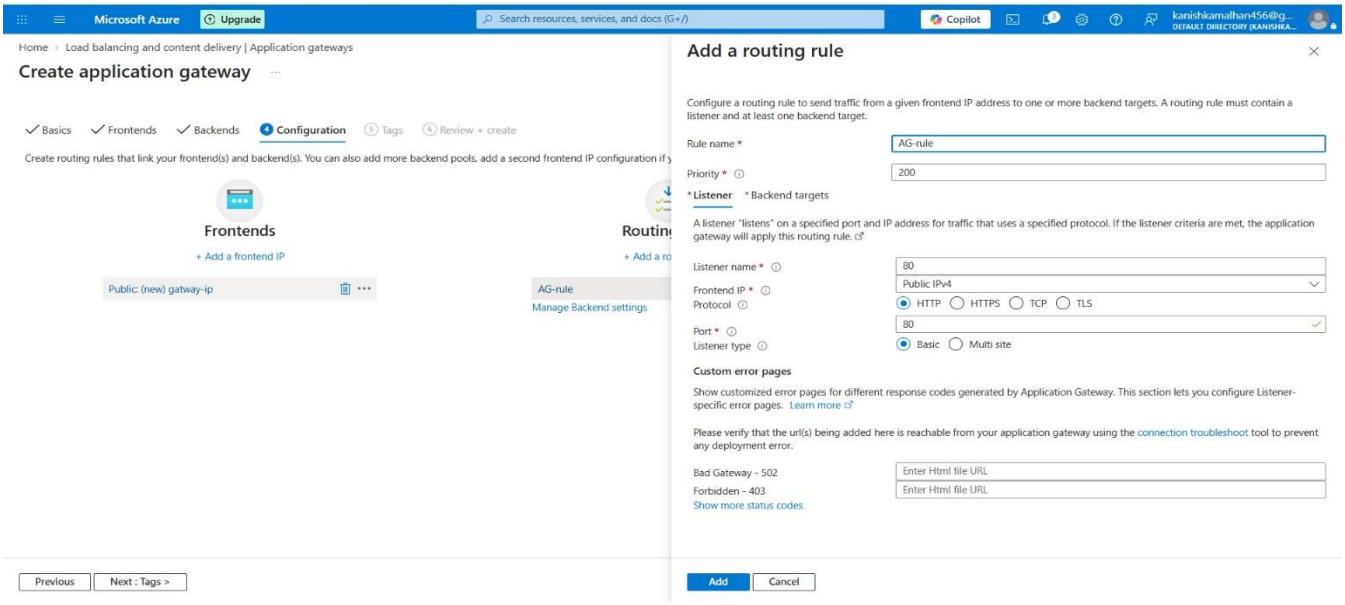


Step 9: Configure Routing Rule

Configured a routing rule to connect the frontend listener and backend pool.

Observation

- Routing Rule configured successfully.
- HTTP traffic configured on Port 80.
- Listener connected successfully with frontend IP.
- Backend routing configured for incoming requests.



Step 10: Complete Application Gateway Deployment

Completed the Application Gateway configuration and deployed the resource.

Observation

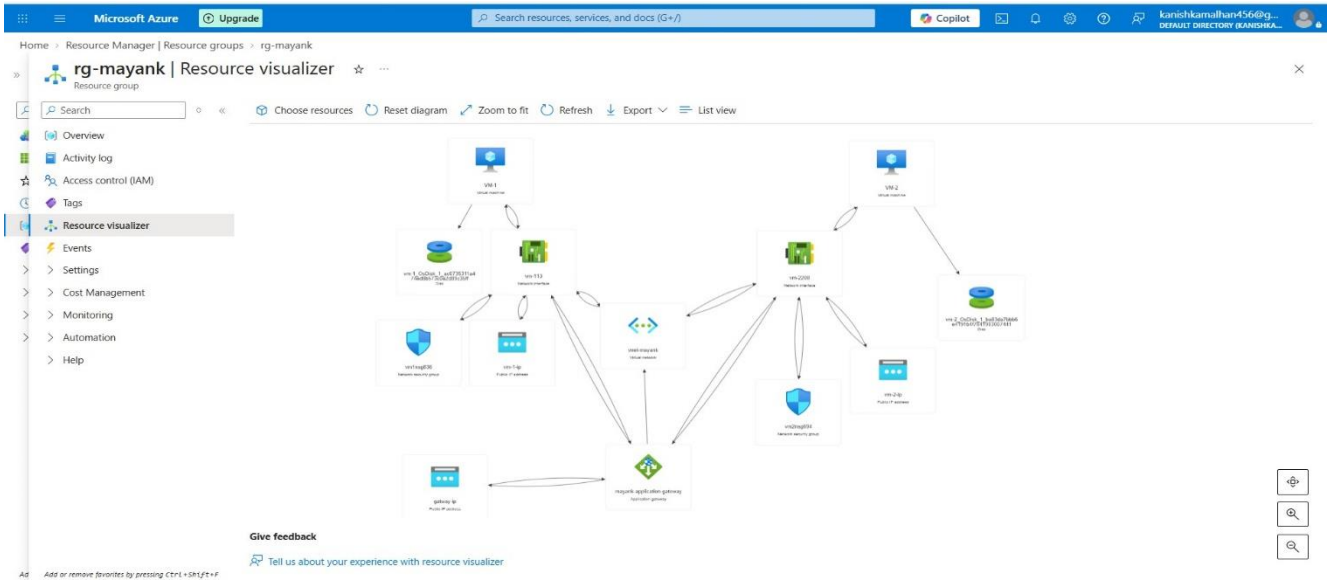
- Frontend configuration completed successfully.
- Backend pool associated successfully.
- Routing rule configured successfully.
- Application Gateway deployed without errors.

Step 11: Validate Resource Connectivity

Used Azure Resource Visualizer to verify resource connectivity and architecture.

Observation

- Application Gateway connected successfully to the virtual network.
- Backend virtual machines associated successfully.
- Frontend Public IP connected successfully.
- Overall architecture verified successfully.



Step 12: Verify Application Access Through Application Gateway

Accessed the Application Gateway Public IP address and validated application availability.

Observation

- Netflix web application loaded successfully through the Application Gateway.
- Requests were successfully routed to one of the backend virtual machines.
- Application Gateway processed incoming traffic successfully.

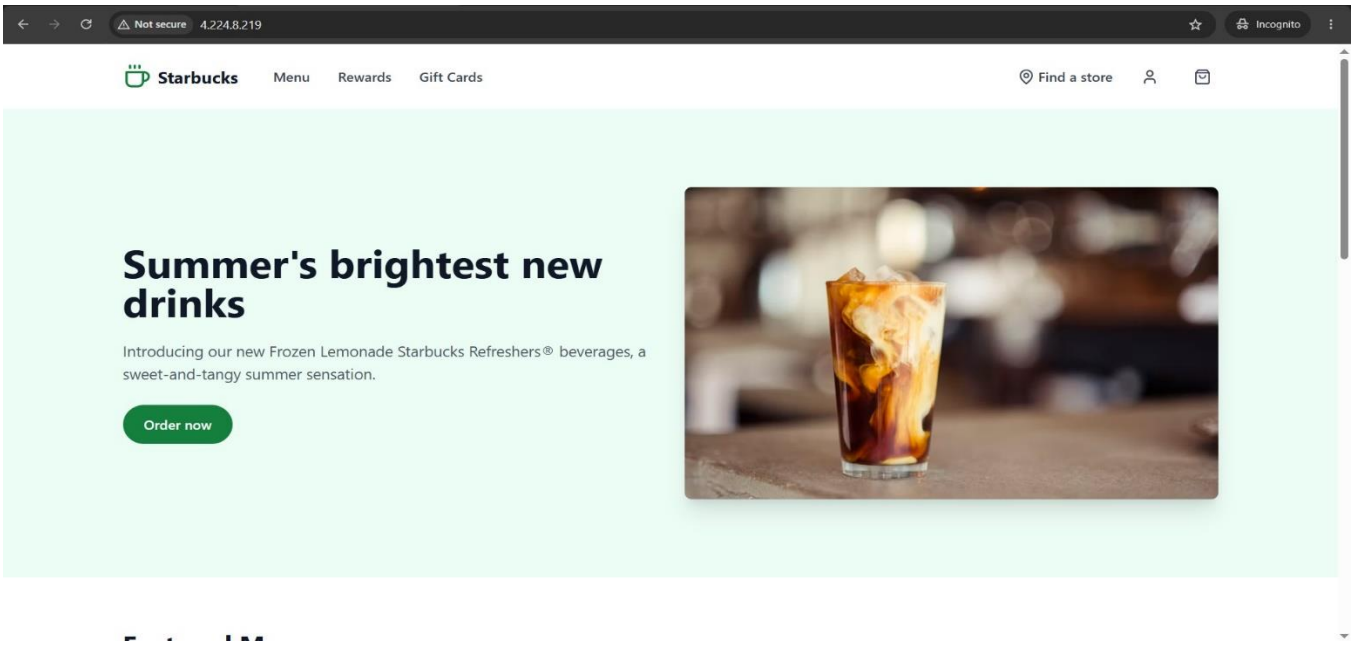


Step 13: Verify Backend Application Accessibility

Accessed the Application Gateway endpoint again and verified traffic routing to another backend application.

Observation

- Starbucks web application loaded successfully.
- Requests were routed successfully to the second backend virtual machine.
- Backend web applications were accessible through the same Application Gateway endpoint.



Result

Successfully configured an Azure Application Gateway for two Ubuntu Linux Virtual Machines running Nginx. Both backend servers were added to a backend pool, routing rules were configured, and incoming HTTP requests were processed through a public frontend endpoint. The Application Gateway successfully routed requests to backend applications hosted on both virtual machines.

Final Observation

- Two Ubuntu Linux Virtual Machines configured successfully.
- Nginx installed and configured on both backend servers.
- GitHub-based web applications deployed successfully.
- Dedicated Application Gateway subnet utilized.
- Azure Application Gateway Standard V2 created successfully.
- Public Frontend IP configured successfully.
- Backend Pool configured with VM-1 and VM-2.
- Routing Rule configured successfully.
- Resource connectivity verified through Resource Visualizer.
- Netflix and Starbucks applications accessed successfully through Application Gateway.
- Web requests routed successfully to backend servers.
- Centralized traffic management achieved using Azure Application Gateway.

Final Result

Azure Application Gateway was successfully configured and incoming web traffic was routed to backend virtual machines hosting different web applications, providing centralized traffic management, improved availability, and efficient application delivery.